ljopt: Translation Validation and IR-Level Fuzzing for the LuaJIT Trace Optimizer*

Aleksey Churkin[1], Sergey Bronnikov[2], and Sergey Kaplun[2]

[1] Moscow Institute of Physics and Technology (MIPT), Dolgoprudny, Russia
churkin.as@phystech.edu
[2] VK, Moscow, Russia

Abstract. We demonstrate ljopt, a translation-validation tool for the LuaJIT tracing JIT compiler: it records the trace a program produces twice, unoptimized and optimized, translates them into one SMT-LIB formula asserting their inequivalence, and lets cvc5 or Z3 prove unsat (equivalent) or return a counterexample. The encoding covers IEEE-754 arithmetic, tagged values, versioned table memory, guards and side-exit snapshots, FFI integers, and strings. On top of the validator we built irfuzz, a fuzzer that synthesizes typed IR instruction streams and replays them through LuaJIT's real fold engine, using the validator itself as the bug oracle. Random and exhaustive enumeration modes reach 97.5 % line coverage of LuaJIT's optimizer, and the oracle's sensitivity is confirmed by fault injection. The tools re-detect six historical LuaJIT miscompilation bugs and found new ones.

Keywords: translation validation · JIT compilation · SMT · compiler fuzzing · LuaJIT

1 Introduction

Tracing JIT compilers [5,3] compile only the hot paths a program actually takes, which makes their optimizers both unusually aggressive and unusually hard to test: the unit of compilation is not a function but a dynamically recorded instruction trace, specialized by guards that fall back to the interpreter. LuaJIT [12] is a mature, widely deployed example—it powers OpenResty, game scripting, and the Tarantool database—yet its optimizer has a steady history of miscompilation bugs, several of which survived in releases for years.

Translation validation [13,11] checks each individual compilation instead of the compiler: rather than proving the compiler correct, prove that *this* optimized program is equivalent to *this* unoptimized one. Alive2 [8] made this practical for LLVM. Tracing JITs are a natural fit: a trace is a straight-line program with explicit guards, so the control flow that makes whole-function validation hard largely disappears.

This paper presents ljopt, a translation validator for LuaJIT traces, and irfuzz, an IR-level fuzzer that turns the validator into a bug-finding oracle. Our contributions:

* Tool demonstration paper.

1. An SMT encoding of LuaJIT IR traces covering IEEE-754 doubles, tagged values as an algebraic datatype, versioned table memory, guards and side-exit snapshots, bounded loop unrolling, FFI 64-bit integers, and strings (Sect. 3). The encoding is solver-portable and runs on both cvc5 [1] and Z3 [10].
2. irfuzz: a fuzzer that synthesizes typed IR instruction streams and replays them through LuaJIT's *actual* fold engine twice (optimizations off and on), then asks the validator whether the two results are equivalent (Sect. 4). Random and exhaustive modes reach 97.5 % line coverage of the optimizer, and fault injection confirms the oracle actually fires.
3. New bugs and a fix: a narrowing miscompile in `lj_opt_narrow.c` found by exhaustive enumeration (we wrote the patch) (Sect. 6). A 35-case corpus of historical LuaJIT bugs, validated sat-on-buggy / unsat-on-fixed (Sect. 7).

2 Background: LuaJIT Traces

When a loop or function becomes hot, LuaJIT records the bytecode it executes into a linear SSA IR. Type checks and control decisions become *guards*: each restores the interpreter state described by the dominating *snapshot*. The optimizer pipeline runs over this IR: the fold engine, alias analysis with load forwarding and dead-store elimination, loop-invariant code motion by peeling, allocation sinking, dead-code elimination, and *narrowing*. After optimization the trace is assembled to machine code.

A miscompilation at this level is observable as a wrong value in a slot, a wrong store to a table, or a guard that exits differently—exactly the three things our encoding compares.

3 Translation Validation for Traces

3.1 Recording twice

ljopt runs the input chunk twice in a sandboxed environment—optimizations off and at `-O3`, with `hotloop=1` so small chunks compile—and captures, for every recorded trace, the full IR with structured operands and all snapshots. Traces from the two runs are matched by their starting location; the pair is then translated into one SMT-LIB formula [2].

3.2 Value and memory model

Lua values are dynamically typed, so cells are modeled as an algebraic datatype with six constructors:

```
(declare-datatypes ((MemCell 0))
  (((int-val (get-bv (_ BitVec 64)))
    (fp-val  (get-fp (_ FloatingPoint 11 53)))
    (str-val (get-str String))
    (tab-val (get-tab Int))
```

```
    (p32-val (get-p32-tab Int) (get-p32-idx MemCell))
    (nil-val))))
(define-sort MemPtr ()
  (Array Int (Array Int (Array MemCell MemCell))))
```

Numbers are IEEE-754 doubles in the SMT floating-point theory [4]; FFI integers are 64-bit bitvectors; strings use the SMT string theory; a table is an integer identity. A *reference into* a table (`p32-val`) carries both the table identity and the key cell, so string and numeric keys can never collide by accident. The formula is quantifier-free throughout.

Memory is a three-level array indexed $[version][table][key]$; every store bumps the version with a functional update, and loads read the current version. Three such arrays exist per trace pair: `shared_mem` is the unknown pre-trace heap, and `mem_unopt`, `mem_opt` are each constrained to equal it at version 0. Both traces therefore start from the *same* arbitrary heap, and the solver searches for any initial state that makes them diverge. Raw FFI memory is a separate byte-addressed array.

3.3 Guards and snapshots

Every guarded instruction contributes a boolean "this guard held." Each snapshot accumulates its guards into a wide bitvector whose least-significant set bit identifies the first failing guard; the formula compares, across the two traces, (i) whether any guard fired at all, and (ii) the restored value of every slot of every snapshot, under a mapping that pairs snapshots across traces by position rather than by index (the optimizer renumbers them). Modeling guard failure as *data*—rather than as an assertion—is essential: guard elimination and guard strengthening are exactly the optimizations one wants to check.

3.4 Floating-point fidelity

The encoding is bit-precise for doubles, with deliberate deviations only where LuaJIT itself deviates from naïve IEEE semantics: `MIN`/`MAX` are encoded as ordered conditionals, not as `fp.min`/`fp.max`, which SMT leaves underspecified on $(+0, -0)$; integer `MOD` is floor-mod; shift counts are masked as on x86; numeric equality uses `fp.eq`, while the final state comparison uses structural equality *relaxed on NaN sign*, because LuaJIT folds $-1.0 \cdot x$ into a sign-bit XOR. $+0.0$ and -0.0 remain distinguished. Transcendentals, `POW`, string–number coercions, and external calls are uninterpreted functions—equivalence is then checked up to congruence.

3.5 Loops, and what "bounded" means

LuaJIT's loop optimization peels one iteration and inserts PHIs. Rather than encode PHIs, ljopt unrolls: the unoptimized trace body n times against the optimized body's peel plus $n-1$ iterations (default n=4), duplicating snapshots per

iteration. Like Alive2, the result is *bounded* translation validation: a divergence that only manifests after more iterations is missed.

Instructions the encoding does not model are handled by *transitive weakening*: the unknown node and everything data-dependent on it are replaced by unconstrained values, and their guards default to "held." This can only weaken the check—it can never invent a divergence—and a strict mode turns any weakening into a hard error. ljopt models roughly 76 of LuaJIT's 82 instruction opcodes via 202 typed handlers.

4 irfuzz: Fuzzing the Optimizer at the IR Level

Translation validation checks the traces your test programs happen to produce. To search the optimizer's input space directly, irfuzz inverts the setup: it *synthesizes* IR and uses the validator as the oracle.

4.1 Replaying synthetic IR through the real optimizer

A 335-line patch adds `jit.util.irfuzz` to LuaJIT: it takes a typed instruction stream (opcodes, result types, operand kinds and payloads) and replays it twice through the *production* fold engine—once with all optimizations disabled and once at `-O3`, optionally followed by the loop, DCE, and sinking passes—returning both resulting IR buffers. No bytecode, recorder, or assembler is involved: the unit under test is precisely the optimizer. A decoder rebuilds each buffer into the exact structure the trace recorder would have produced, so the validator cannot tell synthetic IR from recorded IR, and the comparison entry point is *the same function* used for recorded traces. `unsat` means the optimizer preserved observable state on this input; `sat` is a candidate miscompile or a modeling gap, triaged by replaying the seed.

4.2 Generation

Random mode builds streams of `SLOAD` inputs, constants, and a body of arithmetic, bitwise, comparison, and table operations, using a local deterministic PRNG so seeds reproduce across builds. Operand selection is *blacklist*-based: only combinations the recorder provably never emits, or that hit a known modeling gap, are excluded—a whitelist would only ever exercise what the model already covers and could never surface a new gap. Constants are exactly the literals LuaJIT's fold rules special-case, plus non-special controls.

Exhaustive mode enumerates every operator chain of a given depth over a fixed leaf set. Seventeen targeted enumerators cover specific pass surfaces: conversion chains, guards, aliasing, narrowing, sinking, buffers, bounds checks, upvalues, FFI pointers.

Loop mode feeds generator output i back into the slot that `SLOAD` i reads, making every value loop-carried, and runs LuaJIT's loop pass on the optimized side.

4.3 Solvers

Queries run against standalone solver binaries—with verdicts matched line-exactly. cvc5 is the default. On 180 byte-identical queries drawn evenly from six generation modes, at a 30 s limit, cvc5 closed 166 and Z3 134; the union is 171, with 37 closed only by cvc5 and 5 only by Z3. Making the encoding portable doubled as a soundness review.

4.4 Coverage, and validating the fuzzer itself

A full sweep of all generation modes (8000 traces per mode, with and without loop replay) against a gcov-instrumented LuaJIT covers **2449 of 2513 lines, 97.5 %**, of the optimizer (`lj_opt_fold.c` 97.4 %, `mem` 96.9 %, `loop` 97.6 %, `dce` 100 %, `sink` 98.0 %, `narrow` 98.2 %).

Of the 64 missed lines, 16 are argued unreachable individually (assertion-guarded `default` arms, fold-table entries no rule can reach, arms dead when folding is disabled); each remaining line has a documented reason.

Coverage alone does not show the oracle can *fire*, so irfuzz is validated by mutation: injecting a fault into a fold rule must flip a clean sweep to sat. Injecting an off-by-one into the string-length fold turns a clean sweep (0 sat) into all-sat; forcing the alias analysis to treat may-alias as must-alias flips 18 of 120 aliasing shapes.

5 diffuzz: Differential Testing Below the IR

irfuzz reasons about IR semantics, so it is blind to everything the compiler does after the IR: register allocation, snapshot restoration, trace linking. diffuzz closes that gap from the other side: it generates random Lua programs, runs them under the interpreter and under 16 JIT configurations (optimization levels, hot-counter settings, and each pass individually disabled), and compares outputs.

Every generated value is an exact integer reduced modulo 1000003. Floating point ruins end-to-end differential testing—a one-ULP interpreter/JIT difference is usually a legitimate `libm` difference. The exact-integer generator is also the more sensitive one: against a deliberately injected alias-analysis fault it flags 15 of the first 60 seeds where the float version caught 2.

6 Findings

A new narrowing miscompile (fix authored). The exhaustive narrowing enumeration found that `narrow_conv_backprop` sinks an *unguarded* number-to-integer conversion inside an addition: $int(x + double(i))$ becomes $int(x) + i$, so rounding happens before the addition instead of after. The bug is reachable from plain Lua through every `IRCONV_ANY` consumer:

```
("abcdef"):sub(-0.5 + 3)  --> "cdef"  (interpreter: "bcdef")
bit.tobit(0.5 + 1)        --> 1       (interpreter: 2)
```

We wrote the fix—allow the sunk conversion only for guarded conversion modes—and validate it in CI: the reproducers fail on eight unpatched LuaJIT builds and pass on the patched one.

Other new findings. (i) A wrong loop direction for a `-O` step, reported upstream as LuaJIT issue #1432. (ii) A segmentation fault in `jit.util.tracek()`: the payload half of a 64-bit constant is misread as an instruction, and one byte value turns it into an out-of-bounds dereference; reachable from plain Lua, confirmed on upstream HEAD. (iii) By diffuzz: a store lost at default `-O3`—an `HSTORE` inside a nested loop is dropped by the interaction of five passes (forwarding, DSE, CSE, fold, loop), so a later read observes a stale value. (iv) By diffuzz: with DCE disabled, a loop-invariant value referenced only by side-exit snapshots is assigned a register the loop body reuses, so the exit restores an unrelated variable—a register-allocator bug on which ljopt correctly answers unsat, since the IR is equivalent; the defect is introduced below the IR. This pair of findings is the concrete argument for layering irfuzz and diffuzz.

7 Evaluation

Historical bug corpus. The strongest evidence that the validator's sat verdicts mean something is behavioral: 35 regression cases reconstruct known LuaJIT miscompilation bugs. Each case is run against two pinned LuaJIT builds—one just before the upstream fix, one just after—and the harness requires sat on the buggy build and unsat on the fixed one. Six of these (#794, #797, #817, #1069, #1194, #1236) are detected by the SMT oracle itself, i.e. the encoding sees the semantic divergence; the others reproduce behaviorally and pin the encoding's handling of the affected constructs.

Limitations. ljopt is bounded (loop unrolling, default depth 4) and IR-level: the assembler and register allocator are out of scope by construction, which finding (iv) above makes concrete. Memory is currently compared at its final version only, so a dead store observable at an intervening side exit can be missed—we constructed this false negative deliberately (by weakening DSE's guard check and watching the validator stay unsat) and the fix direction is to compare memory per snapshot at its version.

8 Related Work and Conclusion

Translation validation originates with Pnueli et al. [13] and Necula [11]; Alive and Alive2 [9,8] made bounded SMT-based validation practical for LLVM and are the closest relatives of this work. Compiler fuzzing by differential execution goes back to Csmith [15] and EMI [7]; JIT-directed fuzzers such as Fuzzilli and FuzzJIT [6,14] generate source programs and rely on crashes or differential behavior. irfuzz's distinguishing choice is to generate *IR* and use a translation validator as the oracle, which detects silent wrong-value optimizations that neither crash nor happen to be exercised observably by a source-level test.

Conclusion. ljopt shows that bounded translation validation is a practical fit for a production tracing JIT: the trace shape removes most control-flow complexity, and a careful ADT-plus-versioned-arrays heap encoding handles a dynamically typed language. Used as a fuzzing oracle over synthesized IR, it judges 97.5 % of LuaJIT's optimizer lines and has found and fixed real miscompiles. The tool, the fuzzers, the bug corpus, and all coverage and fault-injection protocols are open source.

References

1. Barbosa, H., Barrett, C., Brain, M., Kremer, G., Lachnitt, H., Mann, M., Mohamed, A., Mohamed, M., Niemetz, A., Nötzli, A., Ozdemir, A., Preiner, M., Reynolds, A., Sheng, Y., Tinelli, C., Zohar, Y.: cvc5: A versatile and industrial-strength SMT solver. In: Tools and Algorithms for the Construction and Analysis of Systems (TACAS). LNCS, vol. 13243, pp. 415–442. Springer (2022)
2. Barrett, C., Fontaine, P., Tinelli, C.: The SMT-LIB standard: Version 2.6. Tech. rep., Department of Computer Science, The University of Iowa (2017), available at `https://smt-lib.org`
3. Bolz, C.F., Cuni, A., Fijałkowski, M., Rigo, A.: Tracing the meta-level: PyPy's tracing JIT compiler. In: Workshop on the Implementation, Compilation, Optimization of Object-Oriented Languages and Programming Systems (ICOOOLPS). pp. 18–25. ACM (2009)
4. Brain, M., Tinelli, C., Rümmer, P., Wahl, T.: An automatable formal semantics for IEEE-754 floating-point arithmetic. In: IEEE Symposium on Computer Arithmetic (ARITH). pp. 160–167 (2015)
5. Gal, A., Eich, B., Shaver, M., Anderson, D., Mandelin, D., Haghighat, M.R., Kaplan, B., Hoare, G., Zbarsky, B., Orendorff, J., Ruderman, J., Smith, E.W., Reitmaier, R., Bebenita, M., Chang, M., Franz, M.: Trace-based just-in-time type specialization for dynamic languages. In: Programming Language Design and Implementation (PLDI). pp. 465–478. ACM (2009)
6. Groß, S., Koch, S., Bernhard, L., Holz, T., Johns, M.: FUZZILLI: Fuzzing for JavaScript JIT compiler vulnerabilities. In: Network and Distributed System Security Symposium (NDSS) (2023)
7. Le, V., Afshari, M., Su, Z.: Compiler validation via equivalence modulo inputs. In: Programming Language Design and Implementation (PLDI). pp. 216–226. ACM (2014)
8. Lopes, N.P., Lee, J., Hur, C.K., Liu, Z., Regehr, J.: Alive2: Bounded translation validation for LLVM. In: Programming Language Design and Implementation (PLDI). pp. 65–79. ACM (2021)
9. Lopes, N.P., Menendez, D., Nagarakatte, S., Regehr, J.: Provably correct peephole optimizations with Alive. In: Programming Language Design and Implementation (PLDI). pp. 22–32. ACM (2015)
10. de Moura, L., Bjørner, N.: Z3: An efficient SMT solver. In: Tools and Algorithms for the Construction and Analysis of Systems (TACAS). LNCS, vol. 4963, pp. 337–340. Springer (2008)
11. Necula, G.C.: Translation validation for an optimizing compiler. In: Programming Language Design and Implementation (PLDI). pp. 83–94. ACM (2000)
12. Pall, M.: The LuaJIT project. `https://luajit.org`, accessed August 2026

13. Pnueli, A., Siegel, M., Singerman, E.: Translation validation. In: Tools and Algorithms for the Construction and Analysis of Systems (TACAS). LNCS, vol. 1384, pp. 151–166. Springer (1998)
14. Wang, J., Zhang, Z., Liu, S., Du, X., Chen, J.: FuzzJIT: Oracle-enhanced fuzzing for JavaScript engine JIT compiler. In: USENIX Security Symposium. pp. 1865–1882 (2023)
15. Yang, X., Chen, Y., Eide, E., Regehr, J.: Finding and understanding bugs in C compilers. In: Programming Language Design and Implementation (PLDI). pp. 283–294. ACM (2011)